The subtree sequence of a tree is primitive recursive

The Open Logic Project contributors

openlogic:computability/recursive-functions:local-snapshot-20261013; CC BY 4.0

Editorial note. The counted statement is the author proposition labelled prop:subtreeseq (2219-2224); the proof span 2229-2261 is the author proof environment that immediately follows it.

Editorial scope. One original result of the Open Logic Project unit computability/recursive-functions, counted once: SubtreeSeq is primitive recursive. Only the counted proposition and its own author proof are retained; the source is reproduced without mathematical changes and keeps the CC BY 4.0 licence and the author attribution.

Allowed standard prerequisites (editorial).

- Coding of trees as sequences. A tree is coded by a sequence whose first component is the number of immediate subtrees, followed by the codes of the immediate subtrees and the root label.. SubtreeSeq returns codes of subtrees of the tree coded by t, and the immediate subtrees are read off the code (2210-2218).
- Primitive recursion. Functions defined from primitive recursive functions by primitive recursion are primitive recursive.. Both the auxiliary g_f and the level helper hSubtreeSeq are defined by primitive recursion (2242-2254).
- Immediate subtrees function. ISubtrees(t) = subseq(t, 1, (t)_0) returns the codes of the immediate subtrees of t and is primitive recursive.. It is the building block whose sequence-images produce the next level of subtrees (2230-2232).

Proposition 0.1. *The function* $\mathrm{fn}\, SubtreeSeq(t)$*, which returns the code of a sequence the elements of which are the codes of all subtrees of the tree with code* t*, is primitive recursive.*

Proof. First note that $\mathrm{fn}\, ISubtrees(t) = \mathrm{fn}\, subseq(t, 1, (t)_0)$ is primitive recursive and returns the codes of the immediate subtrees of a tree t. Now we can define a helper function $\mathrm{fn}\, hSubtreeSeq(t, n)$ which computes the sequence of all subtrees which are n nodes removed from the root. The sequence of subtrees of t which is 0 nodes removed from the root—in other words, begins at the root of t—is the sequence consisting just of t. To obtain a sequence of all level $n + 1$

subtrees of t, we concatenate the level n subtrees with a sequence consisting of all immediate subtrees of the level n subtrees. To get a list of all these, note that if $f(x)$ is a primitive recursive function returning codes of sequences, then $g_f(s,k) = f((s)_0) \frown \cdots \frown f((s)_k)$ is also primitive recursive:

$$\begin{aligned} g(s,0) &= f((s)_0) \\ g(s,k+1) &= g(s,k) \frown f((s)_{k+1}) \end{aligned}$$

For instance, if s is a sequence of trees, then $h(s) = g_{\mathrm{fn}\, ISubtrees}(s, \mathrm{len}\, s)$ gives the sequence of the immediate subtrees of the elements of s. We can use it to define $\mathrm{fn}\, hSubtreeSeq$ by

$$\begin{aligned} \mathrm{fn}\, hSubtreeSeq(t,0) &= \langle t \rangle \\ \mathrm{fn}\, hSubtreeSeq(t,n+1) &= \mathrm{fn}\, hSubtreeSeq(t,n) \frown h(\mathrm{fn}\, hSubtreeSeq(t,n)). \end{aligned}$$

The maximum level of subtrees in a tree coded by t, i.e., the maximum distance between the root and a leaf node, is bounded by the code t. So a sequence of codes of all subtrees of the tree coded by t is given by $\mathrm{fn}\, hSubtreeSeq(t,t)$. □

Proof. First note that $\mathrm{fn}\, ISubtrees(t) = \mathrm{fn}\, subseq(t, 1, (t)_0)$ is primitive recursive and returns the codes of the immediate subtrees of a tree t. Now we can define a helper function $\mathrm{fn}\, hSubtreeSeq(t,n)$ which computes the sequence of all subtrees which are n nodes removed from the root. The sequence of subtrees of t which is 0 nodes removed from the root—in other words, begins at the root of t—is the sequence consisting just of t. To obtain a sequence of all level $n+1$ subtrees of t, we concatenate the level n subtrees with a sequence consisting of all immediate subtrees of the level n subtrees. To get a list of all these, note that if $f(x)$ is a primitive recursive function returning codes of sequences, then $g_f(s,k) = f((s)_0) \frown \cdots \frown f((s)_k)$ is also primitive recursive:

$$\begin{aligned} g(s,0) &= f((s)_0) \\ g(s,k+1) &= g(s,k) \frown f((s)_{k+1}) \end{aligned}$$

For instance, if s is a sequence of trees, then $h(s) = g_{\mathrm{fn}\, ISubtrees}(s, \mathrm{len}\, s)$ gives the sequence of the immediate subtrees of the elements of s. We can use it to define $\mathrm{fn}\, hSubtreeSeq$ by

$$\begin{aligned} \mathrm{fn}\, hSubtreeSeq(t,0) &= \langle t \rangle \\ \mathrm{fn}\, hSubtreeSeq(t,n+1) &= \mathrm{fn}\, hSubtreeSeq(t,n) \frown h(\mathrm{fn}\, hSubtreeSeq(t,n)). \end{aligned}$$

The maximum level of subtrees in a tree coded by t, i.e., the maximum distance between the root and a leaf node, is bounded by the code t. So a sequence of codes of all subtrees of the tree coded by t is given by $\mathrm{fn}\, hSubtreeSeq(t,t)$. □